```
  def cpu
    info = @data_source.get_cpu_info(@id)
    price = @data_source.get_cpu_price(@id)
    result = "Cpu: #{info} ($#{price})"
    return "* #{result}" if price >= 100
    result
  end

  def keyboard
    info = @data_source.get_keyboard_info(@id)
    price = @data_source.get_keyboard_price(@id)
    result = "Keyboard: #{info} ($#{price})"
    return "* #{result}" if price >= 100
    result
  end

  # ...
end
```

Computer is just a wrapper that collects calls, tweaks them a bit, and routes them to a data source. To remove all those duplicated methods, you can turn Computer into a Dynamic Proxy. It only takes an override of method_missing to remove all the duplication from the Computer class.

methods/computer/method_missing.rb

```
class Computer
  def initialize(computer_id, data_source)
    @id = computer_id
    @data_source = data_source
  end

  def method_missing(name)
    super if !@data_source.respond_to?("get_#{name}_info")
    info = @data_source.send("get_#{name}_info", @id)
    price = @data_source.send("get_#{name}_price", @id)
    result = "#{name.capitalize}: #{info} ($#{price})"
    return "* #{result}" if price >= 100
    result
  end
end
```

What happens when you call a method such as Computer#mouse? The call gets routed to method_missing, which checks whether the wrapped data source has a get_mouse_info method. If it doesn't have one, the call falls back to BasicObject#method_missing, which throws a NoMethodError. If the data source knows about the component, the original call is converted into two calls to DS#get_mouse_info and DS#get_mouse_price. The values returned from these calls are used to build the final result. You try the new class in irb: